

P1193R0

Explicitly Specified Returns for (Implicit) Conversions

Published Proposal, 2018-11-26

Author:

JeanHeyd Meneide

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1193.html

Reply To:

[JeanHeyd Meneide](#), [@thephantomderp](#)

Abstract

This paper proposes allowing a user to specify a return type to a conversion operator.

Table of Contents

1 Revision History

1.1 Revision 0 - November 26st, 2018

2 Motivation

2.1 Read-Only

2.2 Mutual Exclusion

2.3 In General

3 Design

3.1 Mechanism

3.2 The Meanings and Syntax

3.3 Opt-In

3.4 Okay, but what if I keep returning things that are convertible?

4 Impact

4.1 On User Code

4.2 On the Standard

5 Proposed Wording and Feature Test Macros

- 5.1 Proposed Feature Test Macro
- 5.2 Intent
- 5.3 Proposed Wording

6 Acknowledgements

References

Informative References

Shared Code	
<pre>template <typename T> auto make_some(std::type_identity<T>) { // for exposition using U = std::remove_const_t<std::remove_reference_t<T>>; return U{}; } template <typename... Args> auto make_some(std::type_identity<std::tuple<Args...>>) { return std::tuple(make_some<Args>()...); }</pre>	
Currently	With Proposal
<pre>struct unicorn_proxy { template <typename T> operator T () { return make_some(std::type_identity<T>()); } }; unicorn_proxy u; int a = 24, b = 3, c = 2; // hard compiler error std::tie(a, b, c) = u; assert(a == 0); assert(b == 0); assert(c == 0);</pre>	<pre>struct unicorn_proxy { template <typename T> auto operator T () { return make_some(std::type_identity<T>()) } }; unicorn_proxy u; int a = 24, b = 3, c = 2; // ta-da!~ std::tie(a, b, c) = u; assert(a == 0); assert(b == 0); assert(c == 0);</pre>
❌ compiler error: <code>could not convert 'make_some(...)'</code> <code>[with Args = {int&, int&, int&}] from 'tuple<int,</code> <code>int, int>' to 'tuple<int&, int&, int&>'</code>	✅ compiles, runs successfully

§ 1. Revision History

§ 1.1. Revision 0 - November 26st, 2018

Initial release.

§ 2. Motivation

There are many types which take advantage of certain conversion operations but need to have their type pinned down exactly in order for such conversions to work properly. We will review two cases here which speak to the heart of the problem: the "Read-Only" effect, and the "Mutual Exclusion" effect.

§ 2.1. Read-Only

A primary example is `std::tie( a, b, c )`, where it generates a tuple of references that expects any left-hand-side of the assignment operation to have types that can assign into each of the reference variables. This works with an explicit conversion:

```
struct fixed_proxy {
    operator std::tuple<int, int, int> () const {
        return std::make_tuple(1, 2, 3);
    }
};

int a, b, c;
// calls conversion operator
std::tie(a, b, c) = fixed_proxy{};
// a == 1, b == 2, c == 3
```

This breaks down when the type for the conversion operation is deduced. Consider a structure that is meant to be converted to anything that appears on the left hand side of an assignment expression or in any kind of constructor (an "omni" or "unicorn" proxy type):

```
struct unicorn_proxy {
    template <typename T>
    operator T () {
        // convert to some T
        // we hardcore this here for example purposes,
        // but usually comes from some make_some<T>()
        // function
        return std::make_tuple(1, 2, 3);
    }
};

int a, b, c;
// compiler error
std::tie(a, b, c) = unicorn_proxy{};
```

This is simply a hard compiler error, because `T` is deduced to be `std::tuple<int&, int&, int&>`. Therefore, it becomes impossible to return newly constructed values into tuple, and effectively locks us out of participating in `std::tie` or similar systems in C++. One would think they could perform some degree of result filtering or SFINAE to allow this to work. But, it does not:

```

struct unicorn_proxy {
    // compiler error
    template <typename T>
    operator remove_internal_tuple_references_t<T> ();
};

```

This is also a hard compiler error, because only a potentially cv-qualified non-dependent type identifier is allowed by the grammar for the so-called "type argument" of a conversion member function.

While developers can still apply SFINAE with `enable_if` and `friends` in the template, we cannot change the the type of `T` itself. This is the essence of the "Read-Only" problem. Developers may query and utilize its properties, but the result -- the thing developers are interested in changing to play nice with `std::tie` and other systems -- is an opaque black box that no one can touch.

§ 2.2. Mutual Exclusion

The mutual exclusion effect is very simple. Consider a type which is interested in the difference between a reference and a value (as is the case for [sol2's proxy types](#)):

```

struct unicorn {
    template <typename T>
    operator T () {
        static std::decay_t<T> v = std::decay_t<T>{};
        return v;
    }

    template <typename T>
    operator T& () {
        static std::decay_t<T> v = std::decay_t<T>{};
        return v;
    }
};

unicorn u;
int i1 = u;
int& i2 = u;

```

The compiler will error here, stating that the conversion is ambiguous and that it cannot choose between either conversion operator:

```

error: conversion from 'unicorn' to 'int' is ambiguous
    int i1 = u;
           ^

```

If the developer attempts to reduce it by removing the second conversion hoping that the first will be able to catch different reference types, the compiler will complain that it cannot initialize `i2` properly:

```
error: cannot bind non-const lvalue reference of
type 'int&' to an rvalue of type 'int'
    int& i2 = u;
           ^
```

This means it is impossible to handle the difference between `int&` and `int` for a single type during a conversion in C++. This happens with templated and non-templated conversion operators.

§ 2.3. In General

In general, C++'s conversion operators pick both the type and the result of an implicit conversion expression without letting the user perform any useful changes that they can normally perform with a regular function. It also does not let a single conversion operation handle different cv-qualified and ref-qualified types, leaving a very useful and specific class of conversions out. There are many cases where loosening the declaration, definition and usage of conversion operators would greatly benefit library and user code.

Therefore, this paper proposes allowing the user to specify the return type of a conversion operation, and for templated conversion operations with an explicitly specified return type to be capable of capturing both a reference and value conversions similar to forwarded template parameters.

§ 3. Design

The primary design goal is to make the feature an entirely opt-in specification that interacts with the language in the same way regular conversions do, just with the compiler no longer assuming the return type is exactly the same as the type argument used to select the conversion operator. Here is an example of the full potential of a templated conversion operation with a changed return type:

```
struct new_unicorn_proxy {
    // capture anything
    template <typename T>
    decltype(auto) operator T&& () {
        // ... return anything
        return make_some<std::remove_reference_t<T>>();
    }
};
```

We go over the set of design decisions made for this extension to the language.

§ 3.1. Mechanism

Allowing an implicit conversion to return different types and deduce reference qualifiers alongside cv-qualifiers opens up a few unique opportunities. The anatomy of this proposal is: `return_type operator type_argument ();`.

§ 3.2. The Meanings and Syntax

Enabling explicit returns comes with a few interesting design decisions when it comes to the syntax and the meanings. Thankfully, the change is wholly conservative and does not complicate or change the grammar with any new keywords or

terminology. There is a difference in semantics, however, which is why it is incredibly important that this feature is [§3.3 Opt-In](#):

```
struct unicorn_value {
    template <typename T>
    auto operator T ();
};

struct unicorn_ref {
    template <typename T>
    auto operator T& ();
};
```

The above two behave like they always do: no matter what you decorate the left hand side of your expression with, it will always deduce `T` to be the type without reference qualifiers. However, with the new syntax we introduce a distinction between the old form and the new form:

```
struct unicorn_anything {
    template <typename T>
    auto operator T&& ();
};
```

This conversion operator in particular does not work with only r-value references as the previous form did: `T` will deduce to exactly the type of the expression on the left hand side, including all cv-qualifiers and reference qualifiers. This only happens when you [§3.3 Opt-In](#) to this feature by adding a return type.

The reason for this departure is as explained before. The [§2.2 Mutual Exclusion](#) problem removes classes of code that care about a single type that can be an l-value, an r-value, or just a plain value in C++ code. By allowing a type argument that has the same capture rules as a forwarding reference, we can capture these differences and act on them in code.

Similarly, allowing us to manipulate the return type more thoroughly allows us to handle the `std::tie` and similar problems. Note that this does not actually change the rules for user-defined conversions as they are now by much: the compiler selects which overload is appropriate by using the type argument -- templated or not -- and passes that return value back. If the return value can construct or be used with what the compiler has selected, that is fine. If it cannot, then it will issue a diagnostic (in the same way that the return type of an overloaded function was used incorrectly).

§ 3.3. Opt-In

Any language feature that wants to minimize potential problems and breakage must be opt-in. The syntax we require for our extension is entirely opt-in, and does not affect previous declarations.

The meaning of old code does not change, and neither does the way it interacts with any of the code currently existing in the codebase. Old code continues to be good code, and this mechanism remains in the standard because it is usually what an individual wants to begin with: it can simply be seen as the compact version of the extension we are attempting to provide. Using the new syntax for an explicit return value does not actually change what, e.g. `T` would deduce to in the above case for the `new_unicorn_proxy`.

§ 3.4. Okay, but what if I keep returning things that are convertible?

This is already banned under current rules: all user-defined conversions to non-built-in types may only go through 1 conversion resolution, otherwise the conversion is ill-formed as defined by [class.conv.fct, clause 4](#). Clause 1 of the same also forbids returning the same type as the object the conversion being performed on or the base class.

The same rule still holds: conversions are not allowed to return the same type as the type the conversion is being invoked on (or any of its base classes). The type returned shall also be usable in the context that it is selected in. This means that a conversion that uses a type argument of `int` must return a type that is convertible to `int` without further user defined conversions (but may invoke additional constructors or other overloaded functions based on the selection).

This rule is stringent to fit how strict the current lookup model is (one and only one user defined conversion to non-built-in type per resolution attempt). There may be room to relax the rules to allow more flexibility if the return type is not identical to the type argument, but this proposal does not explore this avenue at this time. It would be more prudent to establish this feature, and then look at the potential extension of this to behave more closely to something like e.g. `operator->`. However, this kind of change would require a breaking change to existing code since the behavior would change and fundamentally affects the purely opt-in nature of this proposed change.

§ 4. Impact

Since this feature has been designed to be [§3.3 Opt-In](#), the impact is absolutely minimal.

§ 4.1. On User Code

While this introduces an extension to a previous language construct, it thankfully does not break any old code due to its opt-in nature. This is a very important design aspect of this extension syntax: it cannot and should not break any old code unless someone explicitly opts into the functionality. At that point, the potential breakage is still completely bounded, because the return type a developer chooses for a conversion operator member is up to them.

§ 4.2. On the Standard

This does not cause any breakages in the Standard Library or with existing code. No facilities in the standard library would need to use this facility currently.

§ 5. Proposed Wording and Feature Test Macros

This wording section needs help! Any help anyone can give to properly process the wording for this section would be greatly appreciated; this wording is done by the author, who is a novice in parsing and producing Standardese suitable for Core Working Group consumption. The following wording is relative to [\[n4762\]](#).

§ 5.1. Proposed Feature Test Macro

The recommended feature test macro is `__cpp_conversion_return_types`.

§ 5.2. Intent

The intent of this proposed wording is to allow for an explicit return type to be optionally defined on a member conversion operator. In particular, this proposal wants to:

- add grammar to allow for a second `conversion-type-id` to precede the `operator` keyword;
- allow the preceding `conversion-type-id` to be the return type and the mandatory following `conversion-type-id` to be the type argument;
- allow for deduced return types to be explicitly marked with `decltype(auto)` and `auto` by having an explicit return, templated or not;
- create a new feature test macro for detecting if this language functionality exists;
- and, add illustrating examples to aid implementers in the desired outcome of the new feature.

Notably, function and array type names are still not allowed as the `conversion-type-id` following the operator. If it is deemed appropriate to allow function type and array return types so long as the `conversion-type-id` is still within the bounds of class conversion function's clause 3 [[class.conv.fct](#)] restrictions, this can be added in.

§ 5.3. Proposed Wording

Modify §10.3.8.2 [**class.conv.fct**], clause 1 to read as follows:

¹A member function of a class X having no parameters with a declarator-id of `operator` and of the form

conversion-function-id:

`operator conversion-type-id`

conversion-type-id:

`type-specifier-seq conversion-declaratoropt`

conversion-declarator:

`ptr-operator conversion-declaratoropt`

specifies a conversion from X to the type specified by the `trailing conversion-type-id`. Such functions are called conversion functions. A `decl-specifier` in the `decl-specifier-seq` of a conversion function (if any) shall not be neither a defining-type-specifier nor static. The type of the conversion function (`[dcl.fct]`) is “function taking no parameter returning `conversion-type-id`” or “function taking no parameter returning `decl-specifier-seq`”. A conversion function is never used to convert a (possibly cv-qualified) object to the (possibly cv-qualified) same object type (or a reference to it), to a (possibly cv-qualified) base class of that type (or a reference to it), or to (possibly cv-qualified) void.¹¹² [

Example:

```
struct X {
    operator int();
    operator auto() -> short; // error: trailing return type without decl-specifier-seq
};

void f(X a) {
    int i = int(a);
    i = (int)a;
    i = a;
}
```

In all three cases the value assigned will be converted by `X::operator int()`. — *end example*]

[Example:

```
struct X {
    auto operator double() -> int; // OK: decl-specifier-seq allows deduction
    char* operator void*();
};

void f(X a) {
    double di = a; // selects first conversion
    float fi = a; // selects first conversion
    void* from_char_ptr = a; // selects second conversion
    char* char_ptr = a; // error: no matching conversion to char*
}
```

When the `conversion-type-id` and `decl-specifier-seq` are both present, the implementation shall pick the `decl-specifier-seq` as the return type but use the `conversion-type-id` as the selection criteria for the conversion and overloading therein

([\[over.best.ics\]](#), [\[over.ics.ref\]](#)). In this case, the *decl-specifier-seq* of `char*` for the second conversion does not affect overload resolution. — *end example*]

Modify §10.3.8.2 [**class.conv.fct**], clause 6 to read as follows:

⁶ A conversion function template shall not have a deduced return type ([\[dcl.spec.auto\]](#)) [specified by its conversion-type-id without a decl-specifier-seq](#). [*Example*:

```
struct S {
    operator auto() const { return 10; }           // OK
    template<class T>
    operator auto() const { return 1.2; }           // error: conversion function template
    template<class T>
    auto operator T\(\) const { return "bjork"; } // OK
};
```

— *end example*]

Append to §14.8.1 Predefined macro names [**cpp.predefined**]'s **Table 16** with one additional entry:

Macro name	Value
_cpp_conversion_return_types	201811L

§ 6. Acknowledgements

Thank you to Lisa Lippincott for the advice and knowledge on how to solve this problem in an elegant and simple manner.

§ References

§ Informative References

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4762 - Working Draft, Standard for Programming Language C++](#), May 7th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf>